

Gabarito do Professor - SQL Intermediário (Módulo 1, Semana 10)

Sep 28, 2026 · @Alvaro P

Como usar este gabarito

Para cada uma das 9 atividades do caderno, este documento traz a consulta-resposta da tarefa, o resultado esperado, as respostas das perguntas feitas ao vivo e os erros mais prováveis dos alunos. Todas as consultas foram executadas em PostgreSQL 16 sobre a base gerada pelo script `base_pratica_postgres.sql`.

- **Pré-requisitos:** PostgreSQL 13 ou superior, script carregado (o banco `pratica_sql` já criado e as tabelas já populadas).
- **Data de referência:** 30/09/2025. As consultas usam `DATE '2025-09-30'` no lugar de `CURRENT_DATE`.
- **Mais de uma resposta correta:** o que vale é o resultado bater com o "Confira" do caderno. Aceite variações de escrita, como `EXTRACT(YEAR FROM data) = 2025` no lugar de um intervalo de datas.
- **Ordem das linhas:** consultas sem `ORDER BY` podem devolver as linhas em ordem diferente. Isso não é erro.
- **Se os números de um aluno divergem em vários exercícios:** o mais provável é a transação de teste 1201 (atividade 3.3) ainda estar na base. Peça que rode `DELETE FROM transacoes WHERE id = 1201;` ou que reexecute o script inteiro.
- **Divergências do Postgres em relação ao MySQL que aparecem neste gabarito:** o Postgres tem `FULL OUTER JOIN` nativo (o MySQL não tem), não aceita apelido do `SELECT` dentro do `HAVING` (o MySQL aceita) e aceita uma subconsulta no `FROM` sem apelido (o MySQL exige). Essas diferenças estão anotadas nas atividades onde aparecem.

Em cada atividade, a seção **Erros comuns** descreve o sintoma que o aluno verá (mensagem de erro ou número errado), para você reconhecer o problema rapidamente durante a aula.

Dia 1: Sumarização e tratamento de dados

Atividade 1.1: Painel de KPIs por status

Consulta-resposta

```
SELECT status,
       SUM(valor)                AS total,
       AVG(valor)                AS ticket_medio,
       COUNT(DISTINCT cliente_id) AS clientes
FROM   transacoes
WHERE  data >= DATE '2025-01-01' AND data < DATE '2026-01-01'
GROUP BY status
ORDER BY total DESC;
```

Resultado esperado

status	total	ticket_medio	clientes
concluida	73.831,00	245,29	115
pendente	22.256,37	244,58	38
cancelada	12.332,08	241,81	19
estornada	11.868,80	237,38	19

Respostas das perguntas ao vivo

- *Quantas linhas voltam no passo 1? Quatro, uma por status.*
- *Por que "clientes" é tão menor que "ops"? COUNT(*) conta transações e COUNT(DISTINCT cliente_id) conta clientes diferentes; cada cliente compra várias vezes.*
- *Por que COUNT(*) (1.200) e COUNT(valor) (1.176) diferem? Há 24 transações com valor nulo. COUNT(coluna) ignora nulos, e o AVG também: o ticket médio é calculado sobre 1.176 valores, não sobre 1.200.*

Erros comuns

- COUNT(cliente_id) sem DISTINCT: a coluna "clientes" vira 301, 101, 51 e 50, que são os números de transações de 2025, não de clientes.

- Filtrar o período no `HAVING`: dá o erro `column "transacoes.data" must appear in the GROUP BY clause or be used in an aggregate function`. O período se filtra no `WHERE`.
- `EXTRACT(YEAR FROM data) = 2025` no lugar do intervalo: dá o mesmo resultado. Aceite.

Ressalva sobre a base: na base sintética, cada cliente aparece com um único status. Por isso os clientes por status somam o total de clientes distintos. Em dados reais os grupos se sobreporiam; vale avisar a turma para não generalizar essa soma.

Atividade 1.2: Higienização de cadastro

Consulta-resposta

```
SELECT nome,
       TRIM(LOWER(email))                AS email_limpo,
       SPLIT_PART(TRIM(LOWER(email)), '@', 1) AS usuario,
       CONCAT('****', RIGHT(telefone, 4))   AS tel_mascarado
FROM   passageiros
ORDER BY id
LIMIT 10;

SELECT COUNT(*) AS emails_sujos
FROM   passageiros
WHERE  email <> TRIM(LOWER(email));
```

Resultado esperado: as três primeiras linhas são `Passageiro 001 | passageiro1@gmail.com | passageiro1 | ****4729`, `Passageiro 002 | passageiro2@hotmail.com | passageiro2 | ****9458` e `Passageiro 003 | passageiro3@outlook.com | passageiro3 | ****4187`. O total de e-mails sujos é 55 (42 com maiúsculas e 13 com espaços nas pontas).

Respostas das perguntas ao vivo

- *Por que 14 linhas se há só 5 provedores?* Cada domínio aparece em até 3 grafias: minúsculo, MAIÚSCULO e minúsculo com espaço no final. São 5 domínios vezes 3 grafias, menos 1 (o `outlook.com` só tem 2), o que dá 14. Depois de `TRIM(LOWER(...))` sobram 5: `gmail.com` (58), `hotmail.com` (30), `outlook.com` (23), `yahoo.com.br` (21) e `uol.com.br` (18).

- *No passo 2, o que os colchetes e os tamanhos revelam?* Os colchetes mostram os espaços ([passageiro06@hotmail.com]) e o tamanho difere do tamanho_limpo nesses casos. Já nos e-mails em maiúsculas os dois tamanhos são iguais: maiúscula não muda o comprimento. Por isso o WHERE precisa de LOWER além de TRIM.

Erros comuns

- Contar os sujos só com LOWER : dá 42. Só com TRIM : dá 13. O certo é 55.
- Aplicar TRIM(LOWER(...)) só na coluna email_limpo e não no usuario : o usuário sai com maiúsculas ou espaços (por exemplo PASSAGEIR04).
- Usar SUBSTRING_INDEX , que é do MySQL: dá o erro function substring_index(character varying, unknown, integer) does not exist . No Postgres a função é SPLIT_PART .
- Usar os 4 primeiros caracteres do telefone em vez dos 4 últimos: RIGHT pega o final, LEFT o começo.

Atividade 1.3: Faturamento mensal e faixas de valor

Consulta-resposta

```
SELECT CASE
    WHEN DATE_PART('year', AGE(DATE '2025-09-30', data_cadastro)) < 2
    THEN 'menos de 2 anos'
    WHEN DATE_PART('year', AGE(DATE '2025-09-30', data_cadastro)) < 5
    THEN '2 a menos de 5 anos'
    ELSE '5 anos ou mais'
    END AS faixa,
    COUNT(*) AS motoristas
FROM    motoristas
GROUP BY faixa;
```

Resultado esperado

faixa	motoristas
menos de 2 anos	7
2 a menos de 5 anos	20
5 anos ou mais	13

Respostas das perguntas ao vivo

- Passo 1: 21 linhas, de 2024-01 a 2025-09.
- Passo 2: 639 corridas Econômicas, 1.293 Médias e 1.068 Premium.
- Passo 3: a primeira linha é 2024-01, com premium de 6.742,00 e demais de 4.855,60.
- O que o `CASE` sem `ELSE` devolve quando nada casa, e por que o `SUM` não se importa? Devolve `NULL`, e o `SUM` ignora `NULL`. Por isso cada coluna soma só as linhas da sua faixa.

Erros comuns

- Usar `TIMESTAMPDIFF`, que é do MySQL: dá o erro `column "year" does not exist`. A mensagem confunde porque não menciona a função; o Postgres tenta ler `YEAR` como nome de coluna antes mesmo de checar se `TIMESTAMPDIFF` existe. No Postgres, anos completos se calculam com `DATE_PART('year', AGE(data_recente, data_antiga))`, com a data mais recente primeiro e `'year'` entre aspas.
- Calcular a diferença de anos-calendário (`EXTRACT(YEAR FROM DATE '2025-09-30') - EXTRACT(YEAR FROM data_cadastro)`): dá 6, 19 e 15, porque conta a virada do ano e não os anos completos.
- Usar `<=` nos cortes: dá 12, 23 e 5, porque quem tem exatamente 2 ou 5 anos completos cai na faixa errada. Vale reforçar que "2 a menos de 5" exclui o 5.
- `(DATE '2025-09-30' - data_cadastro) / 365` (subtração de datas dá um inteiro de dias no Postgres): nesta base dá o mesmo resultado (7, 20 e 13), então aceite, mas explique que não são anos completos exatos e pode divergir em outras datas por causa dos anos bissextos.
- Esquecer o `GROUP BY faixa`: o Postgres devolve o erro `column "motoristas.data_cadastro" must appear in the GROUP BY clause or be used in an aggregate function`.

Dia 2: Relacionamentos e filtros agregados

Atividade 2.1: Enriquecer a transação com o cadastro

Consulta-resposta

```

SELECT c.segmento,
       COUNT(*)           AS transacoes,
       ROUND(AVG(t.valor), 2) AS ticket_medio
FROM   transacoes t
INNER JOIN clientes c ON c.id = t.cliente_id
WHERE  t.status = 'concluida'
GROUP BY c.segmento
HAVING COUNT(*) > 170
ORDER BY ticket_medio DESC;

```

Resultado esperado

segmento	transacoes	ticket_medio
Varejo	177	249,78
Startup	175	238,45
Corporate	183	237,03

O Premium fica de fora por ter 165 transações concluídas.

Respostas das perguntas ao vivo

- Passo 1 (faturamento por segmento): Corporate 71.106,22; Premium 70.601,00; Startup 70.375,30; Varejo 68.946,30.
- *Para onde foram os 4.848,00 de diferença?* O total da tabela é 285.876,82 e os segmentos somam 281.028,82. A diferença são 20 transações cujo `cliente_id` (9001) não existe em `clientes`. O `INNER JOIN` só devolve linhas com par nas duas tabelas, então elas somem sem aviso. Esse é o gancho para a atividade 2.2.

Erros comuns

- Esquecer o filtro de status: aparecem os 4 segmentos, inclusive o Premium, com tickets de 245,19 (Corporate), 243,51 (Startup), 242,62 (Premium) e 241,07 (Varejo).
- Colocar `t.status = 'concluida'` no `HAVING`: dá o erro `column "t.status" must appear in the GROUP BY clause or be used in an aggregate function`. Status é filtro de linha, então vai no `WHERE`.

Atividade 2.2: Auditoria de integridade com LEFT, RIGHT e FULL JOIN

Consulta-resposta

```

SELECT c.id, c.nome
FROM   clientes c
LEFT JOIN transacoes t
      ON t.cliente_id = c.id
      AND t.data >= DATE '2025-09-30' - INTERVAL '90 days'
WHERE  t.id IS NULL
ORDER BY c.id;

```

Resultado esperado: 58 clientes, começando por **Cliente 001**, **Cliente 002** e **Cliente 003**. São 10 que nunca compraram (ids 191 a 200) e 48 que só compraram antes de 02/07/2025.

Respostas das perguntas ao vivo

- Passo 1: a linha **Sem cadastro** soma 4.848,00, e o total volta a bater com o da tabela inteira (285.876,82).
- Passo 2: 20 transações órfãs, todas do **cliente_id 9001**.
- Passo 3: 10 clientes sem compra (ids 191 a 200). Com **LEFT JOIN** e as tabelas invertidas o resultado é idêntico.
- Passo 4: o **FULL OUTER JOIN** nativo devolve 1.210 linhas diretamente, sem precisar de **UNION**. Vale frisar que essa é uma vantagem real do Postgres sobre o MySQL para esse tipo de auditoria.
- Passo 5: *onde a versão A perdeu 10 linhas?* O **UNION** elimina linhas idênticas em qualquer banco, e o **SELECT** da versão A só tem **nome** e **valor**. Duas transações do mesmo cliente com o mesmo valor viram uma só: são 5 pares plantados (transações 200 e 390, 201 e 391, 202 e 392, 203 e 393, 204 e 394). Outros 5 pares são do mesmo cliente com **valor** nulo, porque o **UNION** considera dois **NULL** iguais. A versão B inclui o **id** (que é único, então nenhuma linha se repete), usa **UNION ALL** e o **WHERE t.id IS NULL** evita repetir o que o **LEFT JOIN** já trouxe.

Erros comuns

- Condição de data no **WHERE**: devolve 0 linhas. Nas linhas sem par, **t.data** é **NULL**, e o **WHERE** elimina justamente elas. Esta é a armadilha central da atividade.
- **INNER JOIN** no lugar do **LEFT JOIN**: também devolve 0, porque o **INNER** já descarta os clientes sem compra.
- Trocar o sentido da pergunta (**t.data < limite**): devolve 190 clientes, os que compraram em algum momento antes da janela, e não os que ficaram sem comprar dentro dela.

- `NOT EXISTS` com subconsulta correlacionada: também dá 58. É uma solução válida do Dia 3; aceite.
- Tentar simular o `FULL OUTER JOIN` com `UNION` em vez de usar o nativo: funciona, mas é mais trabalho e sujeito ao mesmo risco de perder linhas visto no passo 5. Não é erro, mas vale perguntar por que o aluno não usou o `FULL OUTER JOIN` direto.

Atividade 2.3: Duelo WHERE × HAVING

Consulta-resposta

```
SELECT c.segmento,
       SUM(t.valor) AS faturamento
FROM   transacoes t
INNER JOIN clientes c ON c.id = t.cliente_id
WHERE  t.status = 'concluida'
GROUP BY c.segmento
HAVING SUM(t.valor) > 42000
ORDER BY faturamento DESC;
```

Resultado esperado

segmento	faturamento
Varejo	44.211,29
Corporate	43.375,71

Startup (41.728,42) e Premium (41.487,08) ficam de fora.

Respostas do duelo ao vivo

Consulta	O que acontece	Por quê
A: <code>WHERE SUM(valor) > 1000</code>	Erro: <code>aggregate functions are not allowed in WHERE</code>	O <code>WHERE</code> roda antes de existir qualquer soma
B: <code>HAVING valor > 0</code>	Erro: <code>column "transacoes.valor" must appear in the GROUP BY clause or be used in an aggregate function</code>	Depois de agrupar não existe mais um <code>valor</code> individual

Consulta	O que acontece	Por quê
C: WHERE + HAVING SUM(...)	Roda: concluída 73.831,00 e pendente 22.256,37	O WHERE corta linhas e o HAVING corta grupos
D: HAVING total > 20000 (apelido)	Erro: column "total" does not exist	O Postgres, ao contrário do MySQL, não aceita apelido do SELECT dentro do HAVING. Essa é a diferença mais importante do duelo para quem vem do MySQL: a consulta C, com SUM(valor) escrito por extenso, é a forma portátil.

Ordem de execução para fixar: FROM/JOIN, WHERE, GROUP BY, HAVING, SELECT, ORDER BY.

Respostas do bônus (fan-out do JOIN)

- Soma das transações 1 a 400: 95.646,94. Depois do JOIN com os itens: 191.389,02, quase o dobro.
- A causa é que 400 transações viram 800 linhas (cada uma repete por item), e o SUM soma todas as repetições.
- Somando no lado "muitos": 400 transações distintas e 86.481,00 de receita de itens. Nesta base o valor da transação e a soma dos itens não coincidem; é só para mostrar a mecânica.

Erros comuns

- Esquecer o filtro de status: aparecem os 4 segmentos (Corporate 71.106,22; Premium 70.601,00; Startup 70.375,30; Varejo 68.946,30).
- HAVING t.status = 'concluída': dá o erro column "t.status" must appear in the GROUP BY clause or be used in an aggregate function, como na atividade 2.1.
- Selecionar t.status sem colocá-lo no GROUP BY: dá o mesmo tipo de erro, citando a coluna que falta.
- Agrupar por c.segmento e t.status juntos: funciona e devolve as mesmas duas linhas com uma coluna extra de status. Aceite, mas note que o agrupamento passou a ter dois campos.

- **Importante para quem comparar com a turma do MySQL:** a consulta D do duelo (alias no `HAVING`) funcionava no MySQL e dá erro no Postgres. Se um aluno "colar" a solução de outra turma sem testar, é aqui que ela quebra.

Dia 3: Estruturação lógica e modularidade

Atividade 3.1: Subconsultas como filtro

Consulta-resposta

```
SELECT m.nome, COUNT(*) AS corridas
FROM   corridas c
INNER JOIN motoristas m ON m.id = c.motorista_id
GROUP BY m.id, m.nome
HAVING COUNT(*) > (
    SELECT AVG(q.corridas)
    FROM   (SELECT COUNT(*) AS corridas
            FROM   corridas
            GROUP BY motorista_id) q
)
ORDER BY corridas DESC;
```

Resultado esperado: a média é 75 corridas por motorista e 11 motoristas ficam acima dela.

nome	corridas
Motorista 01	298
Motorista 02	163
Motorista 03	134
Motorista 04	115
Motorista 05	107
Motorista 06	99
Motorista 07	94
Motorista 08	87
Motorista 09	84

nome	corridas
Motorista 10	80
Motorista 11	78

Respostas das perguntas ao vivo

- Passo 1: a média das transações é 243,09 e voltam 545 linhas. *A subconsulta roda uma vez ou uma vez por linha?* Uma vez, porque não depende da linha de fora (não é correlacionada).
- Passo 2: o gasto médio por cliente é 1.496,74 e 89 clientes ficam acima. A média considera 191 grupos de `cliente_id`: os 190 clientes com compra mais o cliente órfão 9001. Se a turma perguntar, vale mostrar que excluí-lo mudaria levemente a média.

Erros comuns

- `HAVING COUNT(*) > AVG(COUNT(*))`: dá o erro `aggregate function calls cannot be nested`. Não se aninham agregações; é preciso calcular a média em uma subconsulta separada.
- Usar agregação no `WHERE` (`WHERE valor > AVG(valor)`): dá o erro `aggregate functions are not allowed in WHERE`. Comparação com agregado exige subconsulta.
- Agrupar só por `m.nome`: funciona, porque os nomes são únicos. Aceite.
- **Atenção, diferença do MySQL:** no MySQL, uma subconsulta no `FROM` sem apelido dá erro. **No Postgres não dá erro** — `FROM (SELECT COUNT(*) AS corridas FROM corridas GROUP BY motorista_id)` roda normalmente, sem precisar de `AS q`. Não marque isso como erro se um aluno omitir o apelido; é só menos legível.

Atividade 3.2: Da subconsulta aninhada para a CTE

Consulta-resposta

```

WITH mensal AS (
    SELECT TO_CHAR(data, 'YYYY-MM') AS mes,
           SUM(valor) AS total
    FROM   transacoes
    GROUP BY mes
),
media AS (
    SELECT AVG(total) AS media_mensal
    FROM   mensal
)
SELECT m.mes,
       ROUND(m.total, 2) AS total,
       ROUND(m.total - a.media_mensal, 2) AS diferenca
FROM   mensal m CROSS JOIN media a
WHERE  m.total > a.media_mensal
ORDER BY m.mes;

```

Resultado esperado: 10 dos 21 meses ficam acima da média de 13.613,18.

mes	total	diferenca
2024-03	14.250,71	637,53
2024-05	14.618,44	1.005,26
2024-07	14.526,01	912,83
2024-08	13.916,59	303,41
2024-09	13.824,28	211,10
2024-11	13.673,93	60,75
2024-12	14.768,46	1.155,28
2025-05	14.041,09	427,91
2025-07	14.167,85	554,67
2025-09	13.780,57	167,39

Respostas das perguntas ao vivo

- Passos 1 e 2: a versão aninhada e a com CTE devolvem o mesmo valor, 13.613,18, sobre 21 meses (2024-01 a 2025-09). A diferença está só na legibilidade.

- Por que o `CROSS JOIN` é seguro no passo 3? A CTE `media` tem uma única linha, então o produto cartesiano é $21 \times 1 = 21$ linhas: cada mês só ganha a média ao lado. Com uma CTE de várias linhas, ele multiplicaria as linhas.

Erros comuns

- `WHERE total > AVG(total)` na consulta final: dá o erro `aggregate functions are not allowed in WHERE`. A média precisa vir de uma CTE ou subconsulta.
- Rodar a CTE em uma instrução e consultar `mensal` em outra: dá o erro `relation "mensal" does not exist`. A CTE só existe dentro da instrução em que foi declarada.
- Repetir o `WITH` para a segunda CTE: dá o erro de sintaxe `syntax error at or near "WITH"`. Um único `WITH`, com as CTEs separadas por vírgula.

Atividade 3.3: View para o dashboard e ponte para o Python

Consulta-resposta

```
CREATE VIEW vw_kpi_mensal AS
SELECT TO_CHAR(data, 'YYYY-MM') AS mes,
       SUM(valor)                AS faturamento,
       COUNT(*)                  AS operacoes,
       AVG(valor)                AS ticket_medio
FROM   transacoes
GROUP BY mes;
```

```
import pandas as pd
from sqlalchemy import create_engine

engine =
create_engine("postgresql+psycopg2://USUARIO:SENHA@localhost/pratica_sql")
df = pd.read_sql("SELECT * FROM vw_kpi_mensal ORDER BY mes", engine)
df.plot(x="mes", y="faturamento", kind="line", marker="o")
```

Resultado esperado: a view tem 21 linhas e a primeira é 2024-01, com faturamento de 13.061,88, 58 operações e ticket médio de 233,25. No Python, o DataFrame tem 21 linhas, `df["faturamento"].mean()` dá 13.613,18 (o mesmo valor da atividade 3.2) e 10 meses ficam acima da média.

Respostas das perguntas ao vivo

- Passo 2: com `tot > 70000` voltam Corporate (71.106,22), Premium (70.601,00) e Startup (70.375,30). O Varejo (68.946,30) fica de fora.

- Passo 3: depois do `INSERT` de 5.000,00 para o cliente 4 (Varejo), o Varejo aparece com 73.946,30. A view não guarda cópia dos dados: executa a consulta de novo a cada uso.
- *A view deixou a consulta mais rápida?* Não. Ela só dá nome à consulta; o custo de execução é o mesmo.

Erros comuns

- Rodar o `CREATE VIEW` duas vezes: dá o erro `relation "vw_kpi_mensal" already exists`. Use `CREATE OR REPLACE VIEW` para recriar.
- Esquecer o `DELETE` da transação 1201 no passo 4: o Varejo continua em 73.946,30 e os números dos exercícios seguintes deixam de bater com o "Confira".
- Senha com caracteres especiais (`@`, `/`, `#`) na URL de conexão do Python: a conexão quebra. Proteja a senha com `urllib.parse.quote_plus(senha)`.
- `ModuleNotFoundError`: falta instalar alguma biblioteca do `pip install pandas sqlalchemy psycopg2-binary matplotlib`.
- Instalar `psycopg2` em vez de `psycopg2-binary`: em alguns sistemas operacionais o `psycopg2` puro precisa compilar contra a biblioteca `libpq`, e a instalação pode falhar com erro de compilação. Não reproduzi essa falha no meu teste (rodou liso), mas é um problema conhecido do `psycopg2`; o `psycopg2-binary` evita isso e é a opção recomendada para a sala de aula.
- String de conexão com `mysql+pymysql://`, copiada do material do MySQL: dá erro do SQLAlchemy dizendo que não reconhece o dialeto. O prefixo correto para Postgres é `postgresql+psycopg2://`.